FICHE · S11 2 LES QUATRE ETATS

Semaine 11 · Exercice 2 — Les quatre états

Module ICT 291 · Développer des interfaces utilisateurs (UI)

Durée : 30 minutes · seul · parcours essentiel et avancé

Où sont les fichiers : dans le pack, dossier `02-exercices/s11/2-les-quatre-etats/depart/`

Mandat

Phare affiche ses 30 cartes, mais seulement quand tout va bien. Sur un téléphone dans le métro, le fichier met trois secondes à venir. Après une faute de frappe, il ne vient pas du tout. Un jour, il arrive cassé, ou vide. Votre mission : provoquer chacun de ces cas, noter ce que voit la personne, puis faire en sorte que la page lui dise toujours ce qui se passe.

Pourquoi

Une page qui charge des données a quatre états : **chargement**, **affichée**, **erreur**, **vide**. La version du cours n'en montre qu'un. Ces messages ne sont pas notés à part. Ils servent deux fois : la personne qui attend sait ce qui se passe, et vous, le jour où votre page reste blanche, vous savez où chercher. Ici, vous voyez chaque état de vos yeux avant de le traiter.

Ce que vous saurez faire

- Ralentir le réseau dans les outils du navigateur, pour voir l'état de chargement.
- Attraper une erreur avec `.catch`, et l'afficher dans la page.
- Vérifier le code de statut avec `reponse.ok`, pour qu'un fichier absent devienne une erreur claire.
- Distinguer un fichier absent (`404`) d'un fichier arrivé mais cassé (`200`).

Les mots

- **État** : ce que la page montre à un moment donné du chargement.
- **`.catch`** : « si quelque chose a échoué plus haut, fais ceci ». Il se place à la fin, après les `.then`.
- **`reponse.ok`** : vrai si le code de statut est un succès (`200`), faux sinon (`404`). `fetch` ne compte pas un `404` comme un échec : c'est à vous de le vérifier.
- **`throw new Error(…)`** : « arrête ici et signale une erreur avec ce texte ». Le `.catch` la reçoit.
- **Limitation du réseau** (throttling) : le menu de l'onglet Réseau qui fait semblant d'avoir une connexion lente.

Fichiers fournis

Dans le dossier `depart/` de cet exercice :

- `vide.json` : une pile sans aucune fiche, `[]`.
- `casse.json` : les fiches de Phare, avec une virgule en trop.
- `etats.md` : le tableau à remplir pendant les tests.

Vous continuez dans votre dossier `s11-phare` de l'exercice 1.

Étapes

1. Préparer les tests

Dans le Finder ou l'Explorateur, copiez les trois fichiers de `depart/` dans votre dossier `s11-phare`, à côté de `data.json`. **Dans VS Code**, ouvrez `etats.md`. Ouvrez Phare avec *Open with Live Server*, puis les outils du navigateur (F12).

C'est réussi si : Phare montre ses 30 cartes, et les trois fichiers sont visibles dans la colonne de gauche de VS Code.

2a. Écrire le message de chargement

Dans `app.js`, juste au-dessus de `fetch("data.json")`, ajoutez :

```
zoneEtat.textContent = "Chargement des lieux…";
```

C'est réussi si : après l'enregistrement, la console ne montre aucune ligne rouge, et les 30 cartes s'affichent toujours.

2b. Effacer le message à l'arrivée

Dans `app.js`, dans le second `.then`, au-dessus de `tousLesLieux = donnees;`, ajoutez `zoneEtat.textContent = "";`. Enregistrez.

C'est réussi si : les 30 cartes s'affichent, sans message au-dessus.

2c. Voir le chargement en 3G

Dans l'onglet Réseau, cochez **Désactiver le cache** (Disable cache), puis ouvrez le menu **Pas de limitation** (No throttling) et choisissez **3G**. Rechargez la page. Remplissez la ligne 1 de `etats.md`. Remettez ensuite le menu sur **Pas de limitation**.

C'est réussi si : la ligne 1 de `etats.md` est remplie, dans ses trois colonnes.

3. Le fichier absent

Dans `app.js`, remplacez `fetch("data.json")` par `fetch("data-absent.json")`. Ce fichier n'existe pas : c'est une faute de frappe volontaire. Enregistrez. Remplissez la ligne 2.

C'est réussi si : la ligne 2 est remplie, y compris le code de la colonne Réseau.

4. Attraper l'erreur

Dans `app.js`, à la fin du bloc `fetch`, remplacez la dernière ligne `});` par :

```
  })
  .catch(function (erreur) {
    console.error(erreur);
    zoneEtat.textContent = "Impossible de charger les lieux. Rechargez la page dans un
moment.";
  });
```

`console.error` garde l'erreur dans la console, pour vous. La seconde ligne parle à la personne qui utilise Phare. Enregistrez. Remplissez la ligne 3.

C'est réussi si : la ligne 3 est remplie, et la page montre une phrase au lieu de rester blanche.

5. Vérifier le code de statut

Dans `app.js`, dans le premier `.then`, au-dessus de `return reponse.json();`, ajoutez :

```
    if (!reponse.ok) {
      throw new Error("Fichier introuvable : code " + reponse.status);
    }
```

Le `!` veut dire « pas » : « si la réponse n'est pas un succès, arrête et signale le code ». Enregistrez. Remplissez la ligne 4, puis comparez sa colonne Console avec celle de la ligne 3.

C'est réussi si : la ligne 4 est remplie, et vous pouvez dire à votre voisin laquelle des deux lignes Console aide le plus à trouver la faute.

6. Le fichier cassé

Dans `app.js`, remplacez `fetch("data-absent.json")` par `fetch("casse.json")`. Enregistrez. Remplissez la ligne 5. Puis ouvrez `casse.json` à la ligne citée par la console et trouvez la virgule en trop, sans la corriger.

C'est réussi si : la ligne 5 est remplie, et vous avez trouvé la virgule.

7. La liste vide

Dans `app.js`, remplacez `fetch("casse.json")` par `fetch("vide.json")`. Enregistrez. Remplissez la ligne 6.

Puis, dans le second `.then`, juste sous `tousLesLieux = donnees;`, ajoutez :

```
    if (tousLesLieux.length === 0) {
      zoneEtat.textContent = "Aucun lieu pour le moment.";
      return;
    }
```

`return` arrête la fonction ici : on n'affiche pas de cartes, il n'y en a pas. Enregistrez.

C'est réussi si : la ligne 6 est remplie, puis la page affiche « Aucun lieu pour le moment. ».

8. Retour à la normale

Dans `app.js`, remettez `fetch("data.json")`. Enregistrez. Répondez ensuite aux deux questions en bas de `etats.md`.

C'est réussi si : les 30 cartes reviennent, sans message au-dessus. Ctrl+F (Cmd+F) `fetch(` dans `app.js` trouve une seule ligne, et c'est `fetch("data.json")`.

Réussi si

- Les six lignes et les deux questions de `etats.md` sont remplies avec ce que vous avez vu.
- Avec `data-absent.json`, la page dit qu'elle n'a pas pu charger, et la console donne le code `404`.
- Avec `casse.json`, la page dit qu'elle n'a pas pu charger, et la console cite une ligne de `casse.json`.
- Avec `vide.json`, la page dit qu'il n'y a aucun lieu.
- Avec `data.json`, les 30 cartes s'affichent, sans message.

Si ça bloque

Ce que vous voyez	Ce qui se passe	Ce que vous faites
Pas de menu **3G**	Le menu porte un autre nom selon la version de Chrome.	Choisissez le réglage le plus lent, sauf **Hors connexion** (Offline).
Le message de chargement passe trop vite pour le lire	Le cache est encore actif, ou la limitation n'est pas choisie.	Cochez **Désactiver le cache** et vérifiez le menu, puis rechargez.
Toute la page reste blanche, même le titre	La limitation **Hors connexion** est choisie : Live Server ne répond plus.	Remettez **Pas de limitation** et rechargez.
`Unexpected token '.'` après l'étape 4	Le `;` de la ligne `});` est resté avant `.catch`.	Le bloc se ferme une seule fois, tout à la fin : `})` puis `.catch(…);`.
Le message « Aucun lieu… » n'apparaît pas	Le bloc `if` est dans le premier `.then`, ou au-dessus de `tousLesLieux = donnees;`.	Placez-le dans le second `.then`, juste sous `tousLesLieux = donnees;`.
Le message d'erreur reste avec `data.json`	Live Server n'a pas encore rechargé, ou le nom n'est pas revenu à `data.json`.	Enregistrez à nouveau et vérifiez le nom dans `fetch`.

Avec l'IA

Pas d'IA pour écrire dans cet exercice : vous devez voir chaque état de vos propres yeux. En revanche, collez votre tableau rempli dans un assistant et demandez : « Rends, pour chaque ligne de ce tableau, une phrase qui explique la cause. N'ajoute aucun code. » Comparez ses phrases avec les vôtres.

Pour les rapides

Parcours avancé : mêmes étapes, puis un ou plusieurs de ces points. Ils ne sont pas notés en plus.

- **L'erreur muette. Dans `data.json`**, renommez l'étiquette `"nom"` de la première fiche en `"Nom"`. Enregistrez. Notez ce que montrent la page, la console et l'onglet Réseau, comme une septième ligne. Remettez `"nom"`.
- **Le même test dans Firefox.** Refaites les tests 4 et 5 dans Firefox et comparez les messages de sa console avec ceux de Chrome.
- **Le temps d'attente.** Avec la limitation **3G**, lisez la colonne **Temps** de la ligne `data.json`. Est-ce assez long pour que le message de chargement serve ?